

Projekt do predmetu ISA - Filtrujúci DNS Resolver

Filip Botlo - xbotlo01

15. November 2025

Obsah

1	Úvod do problematiky	2
2	Implementácia	2
2.1	Prehľad tried a ich funkcionality	2
2.1.1	Trieda CLIParser	2
2.1.2	Trieda DNSResolver	2
2.1.3	Trieda DNSProtocol	3
2.2	DNS protokol implementácia	3
2.3	Filtrovacia logika	3
2.4	Rozšírenia aplikácie	3
2.4.1	Verbose mód	3
2.4.2	Štatistiky	3
3	Testovanie	4
3.1	Manuálne testovanie pomocou dig	4
3.1.1	Testovanie A záznamov – povolené domény	4
3.1.2	Testovanie A záznamov – blokované domény	4
3.1.3	Testovanie subdomén	4
3.1.4	Testovanie rôznych typov DNS záznamov	5
3.2	Automatizované testovanie	5
3.2.1	Python testovací skript	5
3.3	Výsledky testov	5
4	Záver	6
5	Literatúra a zdroje	6

1 Úvod do problematiky

DNS (Domain Name System) je hierarchicky organizovaný distribuovaný systém, ktorý slúži na prekladanie doménových mien na IP adresy a naopak. Umožňuje používateľom prístup k webovým stránkam prostredníctvom ľahko zapamätateľných názvov namiesto zložitých číselných IP adries.

Hlavným cieľom projektu bola implementácia vlastného DNS resolvera, ktorý je schopný spracovať DNS dotazy a zároveň na ne aplikovať filtrovaciu logiku blokovaním vybraných domén v textovom súbore.

2 Implementácia

2.1 Prehľad tried a ich funkcionality

Aplikácia je vyvinutá v jazyku C++ s využitím OOP princípov. Hlavné triedy sú navrhnuté s dôrazom na jednoznačne definované zodpovednosti.

2.1.1 Trieda CLIParser

Slúži na spracovanie argumentov príkazového riadka. Obsahuje metódy na parsovanie jednotlivých parametrov, validáciu vstupných údajov a generovanie nápovedy pre používateľov:

- `parse()` – hlavná metóda pre spracovanie argumentov
- `validateArguments()` – kontrola platnosti zadaných parametrov
- `printHelp()` – zobrazenie nápovedy pre používateľa

Umožňuje podporu všetkých nevyhnutných parametrov, vrátane nastavenia portu, upstream DNS servera, cesty k filtrovaciemu súboru, verbose módu a poskytovania štatistických údajov.

2.1.2 Trieda DNSResolver

Funguje ako jadro aplikácie, zabezpečuje koordináciu všetkých operácií a je zodpovedná za vytváranie sieťových socketov, implementáciu serverového cyklu a správu komunikácie medzi klientmi a upstream DNS serverom. Medzi jej kľúčové časti patrí:

- `createClientSocket()` – vytvorenie socketu pre prijímanie dotazov od klientov
- `createResolverSocket()` – vytvorenie socketu pre komunikáciu s upstream DNS
- `loadFilterFile()` – načítanie zoznamu blokovaných domén zo súboru
- `handleQuery()` – spracovanie jednotlivého DNS dotazu
- `run()` – hlavný serverový cyklus

Serverový cyklus používa `select()` s `recvfrom()` pre prijímanie dotazov a `sendto()` pre odosielanie odpovedí.

2.1.3 Trieda `DNSProtocol`

Táto trieda obsahuje všetku logiku súvisiacu s DNS protokolom podľa RFC 1035. Je implementovaná ako statická trieda s utility metódami pre prácu s DNS správami. Hlavné funkcie:

- `parseDomain()` – parsovanie doménových mien z DNS správ
- `isQuery()` – kontrola, či je správa DNS dotazom
- `processQuery()` – hlavná logika filtrovania a spracovania dotazov
- `sendErrorResponse()` – generovanie chybových DNS odpovedí

2.2 DNS protokol implementácia

Domény sú uložené ako sekvencia labels, pričom každý label obsahuje svoju dĺžku a obsah. Resolver dokáže správne dekodovať tieto štruktúry a zrekonštruovať celé doménové meno.

2.3 Filtrovacia logika

Najprv sa doména normalizuje, teda prevod na malé písmená a odstránenie nadbytočných znakov, ako sú koncové bodky. Potom sa vykoná kontrola presnej zhody s blokovanými doménami. Na podporu subdomén bol implementovaný algoritmus, ktorý identifikuje, či normalizovaná doména končí bodkou a blokovanou doménou. Tým je možné blokovať všetky poddomény bez nutnosti uvádzať každú z nich. Filtrovacia logika je obmedzená len na A záznamy (typ 1) podľa zadania, pričom ostatné typy záznamov sú odmietané s chybovým kódom NOTIMP.

2.4 Rozšírenia aplikácie

2.4.1 Verbose mód

Verbose mód je dostupný ako voliteľný parameter `-v`. Pri jeho zapnutí resolver poskytuje podrobné detaily o každom spracovanom DNS dotaze, čím uľahčuje sledovanie aplikácie a prípadné debuggovanie editáciou výstupu. Funkcionalita verbose módu je začlenená do tried `DNSResolver` a `DNSProtocol`, kde sa pred každým diagnostickým výpisom kontroluje flag `verbose_`.

2.4.2 Štatistiky

Rozšírenie štatistík funguje ako voliteľný parameter `--stats`. Táto funkcia zhromažďuje a na konci behu DNS resolvera zobrazuje štatistiky o jeho činnosti. Zaznamenávané metriky zahŕňajú:

- Celkový počet dotazov
- Blokované dotazy
- Preposlané dotazy
- Ostatné typy dotazov (NOTIMP)

- Presné zhody
- Zhody s poddomenami

Implementácia štatistík využíva štruktúru `Statistics` definovanú v `DNSProtocol.hpp`.

3 Testovanie

Testovanie sa uskutočnilo prostredníctvom manuálnych testov využívajúcich nástroj `dig` a automatizovaných testov implementovaných v Python skripte.

3.1 Manuálne testovanie pomocou dig

Ako primárny nástroj manuálneho testovania bol zvolený `dig`. Pre každý test boli stanovené konkrétne očakávané výsledky podľa konfigurácie resolvera.

3.1.1 Testovanie A záznamov – povolené domény

Pri testovaní domén, ktoré nie sú blokové, sa očakávalo, že resolver prijme dotaz, správne ho preposiela na upstream DNS server, zachová pôvodné Query ID, a vráti správnu odpoveď so statusom `NOERROR`. Odpoveď mala obsahovať IP adresu získanú od upstream DNS servera.

Príklad testu:

```
dig @127.0.0.1 -p 5353 google.com
```

Týmto dotazom bolo overené, že doména `google.com`, nezahrnutá na zozname blokovovaných domén, je správne preposlaná na upstream server (8.8.8.8). Očakávaným výsledkom bola správna IP adresa a status kód `NOERROR`.

3.1.2 Testovanie A záznamov – blokové domény

Pri testovaní domén zaradených na zoznam blokovovaných bolo požadované, aby resolver identifikoval príslušnú doménu (podľa presnej zhody alebo subdomén), neposlal dotaz na upstream DNS server, vrátil chybovú odpoveď so statusom `REFUSED` (kód 5) a zachoval Query ID z pôvodného dotazu.

Príklad testu:

```
dig @127.0.0.1 -p 5353 facebook.com
```

Doména `facebook.com` je explicitne uvedená v súbore `blocked_domains.txt`, preto som očakával, že dotaz bude odmietnutý s kódom `REFUSED`.

3.1.3 Testovanie subdomén

Pre test som použil subdoménu `www.facebook.com`, pričom som očakával, že bude blokována, pretože `facebook.com` je v zozname blokovovaných domén.

```
dig @127.0.0.1 -p 5353 www.facebook.com
```

Vrátil sa status kód `REFUSED`, čo potvrdilo, že implementácia správne kontroluje, či dotazovaná doména končí bodkou nasledovanou niektorou z blokovovaných domén.

3.1.4 Testovanie rôznych typov DNS záznamov

Keďže resolver podporuje len A záznamy, pre ostatné typy záznamov som očakával status kód NOTIMP (kód 4).

Test MX záznamov:

```
dig @127.0.0.1 -p 5353 google.com MX
```

Vrátil sa status NOTIMP, pretože MX záznamy (typ 15) nie sú podporované. Rovnaké očakávanie som mal aj pre AAAA záznamy (typ 28), NS, TXT, CNAME a ďalšie typy záznamov.

3.2 Automatizované testovanie

Pre komplexnejšie testovanie som vytvoril Python testovací skript `simple_test.py`, ktorý automatizuje opakujúce sa testy a umožňuje systematické overenie funkcionality.

3.2.1 Python testovací skript

Testovací skript `simple_test.py` pokrýva nasledujúce kategórie testov:

- **Kompilácia a základné funkcie** – overenie úspešnej kompilácie a zobrazenia nápovedy
- **Validácia argumentov** – testovanie rôznych kombinácií argumentov, poradia parametrov, neplatných hodnôt
- **IPv4/IPv6 podpora** – testovanie s rôznymi typmi adries a doménovými menami DNS serverov
- **Spracovanie súborov** – testovanie rôznych formátov koncov riadkov (Linux, Windows, Mac), komentárov, prázdnych riadkov
- **DNS dotazy** – systematické testovanie blokových a povolených domén, subdomén, case sensitivity
- **Rôzne typy dotazov** – overenie správneho odmietania nepodporovaných typov (MX, AAAA, TXT, NS, CNAME, atď.)
- **Chybové scenáre** – testovanie neexistujúcich súborov, neplatných DNS serverov, timeoutov

Skript automaticky spúšťa DNS resolver v pozadí, vykonáva testy a následne vyhodnotí výsledky.

3.3 Výsledky testov

Automatizovaný testovací skript úspešne prešiel všetky testy, čo potvrdilo správnu implementáciu všetkých požadovaných funkcií.

4 Záver

Vďaka projektu som sa naučil pracovať s DNS protokolom na nízkej úrovni a lepšie pochopil jeho štruktúru. Získal som praktické skúsenosti so sieťovým programovaním v C++ a s implementáciou vlastného resolvera.

5 Literatúra a zdroje

1. <https://tools.ietf.org/html/rfc1035>
2. <https://man7.org/linux/man-pages/man2/socket.2.html>
3. <https://man7.org/linux/man-pages/man2/sendto.2.html>
4. <https://man7.org/linux/man-pages/man2/recvfrom.2.html>
5. <https://man7.org/linux/man-pages/man2/select.2.html>
6. <https://man7.org/linux/man-pages/man7/ipv6.7.html>